

Υλοποίηση κωδικοποιητή GSM 06.10 Συστήματα Πολυμέσων

Χρήστος Μάριος Περδίκης 10075 cperdikis@ece.auth.gr

Γιώργος Βακασίρης 9661 vakasiris@ece.auth.gr

Χειμερινό Εξάμηνο 2024-2025

Περίληψη

Αναφορά εργασίας για το μάθημα Συστήματα Πολυμέσων του Τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του ΑΠΘ τη χρονιά 2024-2025. Υλοποιήθηκε κωδικοποίηση φωνής σύμφωνα με το πρότυπο GSM 06.10. Υλοποιήθηκαν και τα τρία επίπεδα της εργασίας. Το πρόγραμμα επίδειξης της υλοποίησης δέχεται στην είσοδο ένα αρχείο wav. Το αρχείο χωρίζεται σε frames των 160 samples και πάνω σε αυτά πραγματοποιούνται οι δύο διαδικασίες pre-processing. Έπειτα το κάθε frame κωδικοποιείται, αποκωδικοποιείται, και τέλος δημιουργείται ένα νέο wav αρχείο. Αυτή είναι η ανακατασκευασμένη φωνή που θα ακούει ο δέκτης.

Εισαγωγή;;;

Αναφορά εργασίας. Υλοποιήθηκε το πρότυπο ETSI GSM 06.10.

1 Προετοιμασία — main.py

Το πρόγραμμά μας τρέχει με την εντολή “python main.py”. Το αρχείο `main.py` περιέχει τη λογική δομή του προγράμματος, από εκεί καλούνται όλες οι συναρτήσεις που υλοποιούν την κωδικοποίηση. Με τη βοήθεια της βιβλιοθήκης `scipy` αποθηκεύουμε όλα τα samples του αρχείου φωνής στο numpy array `audio_data_o`. Μπορεί ο χρήστης του προγράμματος να αλλάξει το όνομα του αρχείου εισόδου σε αυτό το σημείο. Οι διαδικασίες της κωδικοποίησης και της αποκωδικοποίησης πραγματοποιούνται frame ανά frame μέχρι το τέλος του αρχείου φωνής. Ένα frame αποτελείται από 160 samples. Ο αριθμός των επαναλήψεων που θα εκτελεστούν είναι ίσος με τον αριθμό των frames που θα επεξεργαστούν και είναι ίσος με $\lfloor \text{len}(\text{audio_data_o})/160 \rfloor$. Κάθε επανάληψη φορτώνεται το επόμενο frame στο `s_new`, numpy array με σταθερό μέγεθος 160 στοιχείων. Η μεταβλητή `offset` λειτουργεί σαν δείκτης στο πρώτο στοιχείο του επόμενου frame. Πάνω στα στοιχεία του `s_new` γίνεται η διαδικασία του preprocessing. Πρώτα καλούμε την συνάρτηση `offset_compensation()` και μετά την `pre_emphasis` για να λάβουμε το array `s`. Η υλοποίηση των δύο συναρτήσεων υπάρχει στο αρχείο `preprocessing.py`. Στα στοιχεία του array `s` η κωδικοποίηση και μετά την αποκωδικοποίηση λαμβάνουμε ένα ανακατασκευασμένο `s0`. Στο τέλος κάθε επανάληψης ενημερώνουμε το αποθηκευμένο short term residual του προηγούμενου frame με το short term residual του τωρινού frame (`prev_frame_st_residual = curr_frame_st_residual`). Όταν ολοκληρωθούν όλες οι επαναλήψεις, κατασκευάζεται ένα νέο αρχείο ήχου “reconstructed_audio.wav” στο οποίο είναι ακουστή η παραμόρφωση που εισήγαγε η διαδικασία της κωδικοποίησης. Ο χρήστης του προγράμματος μπορεί να αλλάξει το όνομα του αρχείου εξόδου σε αυτό το σημείο.

Αν περισσέψουν samples του αρχείου φωνής εισόδου (ο αριθμός samples του δεν διαιρείται ακριβώς με το 160), τότε αυτά τα samples δεν συμμετέχουν στη διαδικασία της κωδικοποίησης και αγνοούνται. Θεωρούμε ότι είναι αποδεκτό να κόψουμε στη χειρότερη περίπτωση $159 \text{ samples}/8\text{kHz} = 19.875\text{ms}$ ήχου παρά να αυξήσουμε την περιπλοκότητα του κώδικά μας.

2 Short Term Analysis — 1ο επίπεδο

2.1 Κωδικοποιητής

Η υλοποίηση του κωδικοποιητή βρίσκεται στο αρχείο `encoder.py`. Η συνάρτηση αυτοσυσχέτισης υπολογίζεται σύμφωνα με την παράγραφο 3.1.4 του προτύπου ως:

$$r_s = \sum_{i=k}^{160} s[i] * s[i-k] \quad (1)$$

όπου **s** ένα array που περιέχει τα 160 samples του frame εισόδου και **rs** array με τις 9 τιμές της αυτοσυσχέτισης. Σχηματίζουμε τους πίνακες:

$$r = \begin{bmatrix} r_s[1] \\ r_s[2] \\ r_s[3] \\ r_s[4] \\ r_s[5] \\ r_s[6] \\ r_s[7] \\ r_s[8] \end{bmatrix}, R = \begin{bmatrix} r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] & r_s[7] \\ r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] \\ r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] \\ r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] \\ r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] \\ r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] \\ r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] \\ r_s[7] & r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] \end{bmatrix} \quad (2)$$

Λύνουμε τις κανονικές εξισώσεις $Rw = r$ ως προς w με τη συνάρτηση `numpy.linalg.solve`. Ο πίνακας w περιέχει τους 8 βέλτιστους συντελεστές $[a_1, a_2 \dots]$ του short term προβλέπτη. Πρέπει όμως να χρησιμοποιήσουμε τους συντελεστές που θα γνωρίζει ο αποκωδικοποιητής. Σχηματίζουμε ως είσοδο της βοηθητικής συνάρτησης `hw_utils.polynomial_coeff_to_reflection_coeff` το array **a_mod**:

$$a_mod = [1, -w[0], -w[1], -w[2], -w[3], -w[4], -w[5], -w[6], -w[7]] \quad (3)$$

Με τους συντελεστές ανάκλασης k_r υπολογίζουμε τα Log-Area Ratios (παράγραφος 3.1.6 εξίσωση 3.4 του προτύπου), τα κβαντίζουμε (παράγραφος 3.1.7) και κατόπιν τα αποκβαντίζουμε (παράγραφος 3.1.8). Για τον κβαντισμό και τον αποκβαντισμό των Log-Area Ratios υλοποιήθηκαν και οι βοηθητικές συναρτήσεις **Nint**, **A** και **B**, ο ορισμός τους βρίσκεται στο τέλος του αρχείου `encoder.py`. Μετατρέπουμε τα αποκωδικοποιημένα Log-Area Ratios σε αποκωδικοποιημένους συντελεστές ανάκλασης. Καλούμε την συνάρτηση `hw_utils.reflection_coeff_to_polynomial_coeff` με είσοδο το array **krd** των αποκωδικοποιημένων συντελεστών ανάκλασης, για έξοδο παίρνουμε τους αποκωδικοποιημένους συντελεστές του προβλέπτη **akd** σε μορφή παρόμοια με αυτή στην [εξίσωση 3](#), δηλαδή:

$$akd = [1, -a_d[0], -a_d[1], -a_d[2], -a_d[3], -a_d[4], -a_d[5], -a_d[6], -a_d[7]] \quad (4)$$

Τέλος λαμβάνουμε το residual **curr_frame_st_residual** εκτελώντας τη συνέλιξη μεταξύ των αρχικών samples **s** και της εξόδου του array **akd**, δηλαδή $d = s * akd$. Αυτό προκύπτει από το βήμα (β) 1 της παραγράφου 2.1.2 της εκφώνησης της εργασίας αλλά και από την παρακάτω απόδειξη:

$$\begin{aligned} \hat{s}(n) &= \sum_{k=1}^8 a_k s(n-k) \\ &= a_1 s(n-1) + a_2 s(n-2) + \dots \\ d(n) &= s(n) - \hat{s}(n) \\ &= s(n) - a_1 s(n-1) + a_2 s(n-2) + \dots \\ &= \sum_{k=0}^8 a'_k s(n-k) \end{aligned}$$

όπου $a'_k = akd$, το οποίο ισχύει.

Δεν υλοποιήθηκε interpolation των τιμών των Log-Area Ratios στον κωδικοποιητή μας.

Κανονικά σαν είσοδο του κωδικοποιητή θέλουμε και το `prev_frame_st_resd`, παρόλο που δεν το χρησιμοποιούμε, τι κάνουμε με αυτό;

Ο κωδικοποιητής του 1ου επιπέδου επιστρέφει με τη σειρά

- τα κωδικοποιημένα **LARc**,

- το `d_reconstruct` ως `curr_frame_st_residual`
- τη λίστα `N`
- τη λίστα κβαντισμένων `bc`
- το long term residual `e` ως `curr_frame_ex_full`

2.2 Αποκωδικοποιητής

Θα έρθει η ώρα του...

3 Long Term Analysis — 2ο επίπεδο

Αλλαγές σε εισόδους και εξόδους συναρτήσεων; Υπάρχουν καν αλλαγές;

3.1 Κωδικοποιητής

Ο κώδικας που περιγράφουμε στην [υποενότητα 3.1](#) ακολουθεί την υλοποίηση του short term analysis στον κωδικοποιητή του 1ου επιπέδου. Η πρόβλεψη του pitch θα γίνει από το array `prev_d`. Είναι διαφορετικό για κάθε subframe και αποτελείται συνολικά από 120 στοιχεία. Στα πρώτα τρία subframes, επιτρέπουμε η πρόβλεψη να γίνει από το (ανακατασκευασμένο) short term residual του προηγούμενου frame. Πιο συγκεκριμένα και λαμβάνοντας υπόψιν ότι $N \in [40, 120]$, όταν βρισκόμαστε στο πρώτο subframe:

$$prev_d = prev_frame_st_residual[40 \dots 160] \quad (5)$$

δηλαδή το `prev_d` αποτελείται από τα τρία τελευταία subframes του προηγούμενου frame. Εφόσον η πρόβλεψη γίνεται από subframes παρελθοντικού frame, αυτά είναι ήδη ανακατασκευασμένα. Όταν βρισκόμαστε στο 2ο subframe:

$$prev_d = prev_frame_st_residual[80 \dots 160] + curr_frame_st_residual[0 \dots 40] \quad (6)$$

το `prev_d` αποτελείται από τα δύο τελευταία subframes του προηγούμενου frame και ακολουθεί το πρώτο — ανακατασκευασμένο πλέον — subframe του τωρινού frame. Με παρόμοιο τρόπο, στο 3ο subframe το `prev_d` αποτελείται από το τελευταίο subframe του προηγούμενου frame και τα δύο πρώτα subframes του τωρινού frame και στο 4ο subframe αποτελείται από τα 3 τρία προηγούμενα subframes του τωρινού frame. Προφανώς τα 40 τελευταία στοιχεία του `prev_d` δεν θα επιλεγθούν ποτέ από την πρόβλεψη εφόσον $N \geq 40$, υπάρχουν για να απλουστεύσουν τη διαχείριση των indices.

Καλούμε τη συνάρτηση `RPE_subframe_sl_tle()` με εισόδους το τωρινό subframe `d` και το `prev_d` για να υπολογίσουμε τα `b`, `N`. Υπολογίζουμε τη συνέλιξη:

$$R(\lambda) = \sum_{i=0}^{39} d[i] * prev_d[120 + i - \lambda], \quad \lambda \in [40, 120] \quad (7)$$

Τα indices των `d`, `prev_d` προκύπτουν από την εξίσωση 3.9 του προτύπου. Για το `d`, $k_o = 0$ (όπου k_o το index του πρώτου στοιχείου του τωρινού frame) και αναφέρεται ήδη μόνο στο τωρινό subframe, άρα το $j * 40$ είναι περιττό. Για το `prev_d`, το $k_j = 120$ πάντα, εφόσον το `prev_d` είναι ουσιαστικά ένα sliding window που για κάθε τωρινό subframe περιέχει τα τρία προηγούμενα subframes. Ελέγχουμε ποιά $R(\lambda)$ είναι η μέγιστη και θέτουμε ως `N` το λ που τη μεγιστοποίησε. Έπειτα υπολογίζουμε το `b` σύμφωνα με την εξίσωση 3.11 του προτύπου. Ο αριθμητής του `b` είναι ίσος με $R(N)$ και ο παρανομαστής του $\sum_{i=0}^{39} (prev_d[120 + i - N])^2$.

Τα `N`, `b` που επιστρέφει η `RPE_subframe_sl_tle()` αποθηκεύεται το καθένα σε μια λίστα τεσσάρων στοιχείων $N[j]$, $b[j]$ ένα για κάθε διαφορετικό subframe. Κβαντίζουμε τα $N[j]$, $b[j]$ (κάθε φορά μόνο του τωρινού subframe). Το $N[j]$ είναι ήδη ένας ακέραιος οπότε είναι ήδη κβαντισμένος. Το $b[j]$ κβαντίζεται σύμφωνα με την εξίσωση 3.15 του προτύπου σε `bc[j]`. Υλοποιήσαμε τις βοηθητικές συναρτήσεις `QLB()` και `DLB()`, όπως αυτές περιγράφονται από τον πίνακα 3.3 του προτύπου. Οι `QLB()` και `DLB()` βρίσκονται στο τέλος του αρχείου `encoder.py`.

Για να υλοποιήσουμε το στάδιο της πρόβλεψης αρχικά αποκβαντίζουμε το **bc[j]** σε **bd[j]** με τη βοήθεια της QLB. Υπολογίζουμε την πρόβλεψη **d_predict**:

$$d_predict[i] = \sum_{j=0}^{39} bd[j] * prev_d[120 + i - N[j]] \quad (8)$$

και το long term residual **e**:

$$e[j * 40 + i] = d[i] - d_predict[i] \quad (9)$$

Το **e** εμπεριέχει το σφάλμα και για τέσσερα subframes του frame, για αυτό είναι αναγκαία η προσθήκη του offset $j * 40$, όπου j προσδιορίζει σε ποιο subframe βρισκόμαστε (για $j = 0$ βρισκόμαστε στο πρώτο subframe του τωρινού frame, για $j = 1$ στο δεύτερο κλπ.). Τέλος, υπολογίζουμε το ανακατασκευασμένο short term residual **d_reconstruct** από το **e**:

$$d_reconstruct[j * 40 + i] = e[j * 40 + i] + d_predict[i] \quad (10)$$

Εφόσον δεν υλοποιούμε το excitation computation στο 2ο επίπεδο και δεν κβαντίζουμε το **e**, το **d_reconstruct** ουσιαστικά σχεδόν ταυτίζεται με το **curr_frame_st_residual**. Χάνουμε μόνο πληροφορία από το **d_predict** λόγω της κβάντισης του **b**. *Ισχύει αυτό;;;*

Μήπως να σταματήσουμε να επιστρέφουμε το curr_frame_st_residual στο 2ο επίπεδο;

Ο κωδικοποιητής του 2ου επιπέδου επιστρέφει με τη σειρά

- τα κωδικοποιημένα **LARc**,
- το **d_reconstruct** ως **curr_frame_st_residual**
- τη λίστα **N**
- τη λίστα κβαντισμένων **bc**
- το long term residual **e** ως **curr_frame_ex_full**

3.2 Αποκωδικοποιητής

Σε αντίθεση με τον κωδικοποιητή, η υλοποίηση του 2ου επιπέδου στον αποκωδικοποιητή βρίσκεται πριν την υλοποίηση του 1ου επιπέδου. Η είσοδος του αποκωδικοποιητή είναι όλες οι έξοδοι του κωδικοποιητή μαζί με το **prev_frame_st_residual**. Ξεκινάμε αποκβαντίζοντας το **b** με την βοηθητική συνάρτηση QLB() (βρίσκεται και στο τέλος του decoder.py). Το **N** είναι ήδη αποκβαντισμένο εφόσον είναι ακέραιος. Ξανακατασκευάζουμε το **prev_d** όπως περιγράψαμε στην [υποενότητα 3.1](#) αυτής της αναφοράς. Υπολογίζουμε το **d_predict** από την [εξίσωση 8](#), και έπειτα το **d_reconstruct** από την [εξίσωση 10](#), με τη μόνη διαφορά ότι χρησιμοποιούμε το **curr_frame_ex_full** αντί για το **e**. Τέλος, θέτουμε **curr_frame_st_residual = d_reconstruct**, έτσι ώστε ο αποκωδικοποιητής του 1ου επιπέδου να χρησιμοποιήσει το ανακατασκευασμένο short term residual.